

Aavash Kuikel
Independent Study
Dr. Zhang
Fisk University

WEEKLY PROGRESS REPORT

Weeks 3 & 4

Project Overview

- **Project Title:** Extending AI-Generated Source Code Detection to Next-Generation LLMs
- **Status:** Pipeline Built — Data Generation In Progress

Overview

Weeks 3 and 4 focused on building the complete data pipeline from scratch. This included writing all 6 pipeline scripts, resolving multiple compatibility issues with the Python 3.13 environment and newer API versions, and successfully generating AI code from all three new LLMs.

Week 3 — Pipeline Development and Environment Setup

Scripts Written:

- **generateCode.py** — queries Claude Haiku, GPT-4o, and Gemini 2.5 Flash for 164 solutions each via their respective APIs
- **prepareDataset.py** — merges AI-generated code with human-written HumanEval-X solutions into labeled CSVs (label 0 = AI, label 1 = human)
- **generateAST.py** — generates Abstract Syntax Tree sequences from code using tree-sitter, adds variable renaming for normalization
- **generateEmbeddings.py** — generates CodeBERT embeddings (768-dim) for code and AST sequences, creates 5-fold train/test splits

- **runClassifier.py** — runs logistic regression classifier with cross-validation, outputs F1/accuracy metrics and comparison table
- **buildGrammar.py** — compiles tree-sitter grammars for Python, Java, C++ into build/my-languages.so

Key Code: generateCode.py — Querying New LLMs

The core generation script queries each LLM using the same prompt style from the original paper's LLM_Prompts.txt file. If one API call fails, the error is logged and the script continues. Already-completed models are automatically skipped on re-run.

```
def generate_claude(problems):
    import anthropic
    client = anthropic.Anthropic(api_key=ANTHROPIC_API_KEY)
    results = []
    for problem in tqdm(problems):
        try:
            message = client.messages.create(
                model="claude-haiku-4-5-20251001",
                max_tokens=1024,
                messages=[{"role": "user", "content": build_prompt(problem)}]
            )
            ai_code = message.content[0].text.strip()
        except Exception as e:
            ai_code = None # log error, keep going
            results.append({'task_id': problem['task_id'], 'ai_code': ai_code})
            time.sleep(0.3) # rate limiting
    return pd.DataFrame(results)

# Same structure used for GPT-4o and Gemini 2.5 Flash
# All 3 model functions run sequentially in __main__
```

Figure 1 — generateCode.py: LLM code generation with fault tolerance and automatic skip on re-run

Key Code: generateAST.py — Structural Feature Extraction

Before generating ASTs, all variable names are renamed to generic identifiers (var_0, var_1, func_0) to remove naming style bias. This forces the embedding model to learn purely structural patterns rather than memorizing common AI naming conventions.

```

# Step 1: Rename all variables to remove naming bias
def rename_variables(code):
    tree = python_parser.parse(bytes(code, 'utf8'))
    return RV_python(tree, code) # var_0, var_1, func_0...

# Step 2: Generate AST token sequence from renamed code
def generate_ast(code):
    tree = python_parser.parse(bytes(str(code), 'utf8'))
    ast_tokens = F_python(tree.root_node, bytes(str(code), 'utf8'))
    return ' '.join(ast_tokens) # e.g. 'module def parameters ...'

# Applied to every row in the merged CSV:
df['new_code'] = df['code'].apply(lambda c: rename_variables(str(c)))
df['ast'] = df['new_code'].apply(lambda c: generate_ast(str(c)))

```

Figure 2 — generateAST.py: Variable normalization + tree-sitter AST token extraction

Key Code: generateEmbeddings.py — CodeBERT Vectors

CodeBERT's CLS token (the first token of the last hidden state) is used as a fixed-length 768-dimensional embedding for each input. Three embedding types are produced: code-only, AST-only, and combined (1536-dim). These become the feature matrix for the classifier.

```

MODEL_NAME = 'microsoft/codebert-base'
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModel.from_pretrained(MODEL_NAME).to(device)

def get_embedding(text: str) -> np.ndarray:
    inputs = tokenizer(text, return_tensors='pt',
                       truncation=True, max_length=512).to(device)
    with torch.no_grad():
        outputs = model(**inputs)
    # CLS token = index 0 of last hidden state
    return outputs.last_hidden_state[:, 0, :].squeeze().cpu().numpy()

# Three feature types per sample:
code_emb = get_embedding(row['code']) # 768-dim
ast_emb = get_embedding(row['ast']) # 768-dim
combined_emb = np.concatenate([code_emb, ast_emb]) # 1536-dim

```

Figure 3 — generateEmbeddings.py: CodeBERT CLS token extraction producing 768-dim feature vectors

Key Code: runClassifier.py — 5-Fold Cross-Validation

Logistic regression with StandardScaler normalization is evaluated using 5-fold stratified cross-validation to ensure balanced class distribution in each fold. The same classifier is run independently for each embedding type to measure each type's contribution.

```
def run_kfold_classifier(df, emb_type, n_splits=5):
    # Select only columns for this embedding type
    X = df.loc[:, df.columns.str.startswith(emb_type)].values

    y = df['actual_label'].values
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
    fold_metrics = []
    for train_idx, test_idx in skf.split(X, y):
        X_train, X_test = X[train_idx], X[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]
        scaler = StandardScaler()
        X_train = scaler.fit_transform(X_train) # normalize train
        X_test = scaler.transform(X_test) # apply same scale to test
        clf = LogisticRegression(max_iter=1000, C=1.0, random_state=42)
        clf.fit(X_train, y_train)
        fold_metrics.append(calculate_metrics(y_test, clf.predict(X_test)))
    # Average all metrics across 5 folds
    return {k: round(np.mean([m[k] for m in fold_metrics]), 4) for k in
            fold_metrics[0]}
```

Figure 4 — runClassifier.py: 5-fold stratified CV with logistic regression; emb_type = 'code_', 'ast_', or 'combined_'

Technical Challenges Resolved:

- HuggingFace dataset loader broke for THUDM/humaneval-x — fixed by downloading dataset directly from GitHub as a .jsonl.gz file
- Anthropic model name outdated (claude-3-5-sonnet-20241022 returned 404) — updated to claude-haiku-4-5-20251001
- Google Gemini library deprecated (google-generativeai) — migrated to new google-genai SDK with genai.Client()
- Gemini 2.0 Flash unavailable for new users — updated to gemini-2.5-flash
- Tree-sitter grammar version incompatibility (version 15 vs required 13-14) — re-cloned specific older compatible versions
- CodeT5+ incompatible with transformers 5.3.0 on Python 3.13 — switched to CodeBERT which has full compatibility
- Google API free tier quota exhausted — enabled billing on Google Cloud project to unlock paid tier

API Keys and Access:

- Obtained Anthropic API key with \$5 credit — Claude Haiku model successfully accessible
- Obtained OpenAI API key with \$5 credit — GPT-4o model successfully accessible
- Obtained Google API key — Gemini API enabled on Google Cloud project with billing

Week 4 — Data Generation and Embedding

Data Generation Results:

- Successfully generated 164 Python solutions from Claude Haiku using claude-haiku-4-5-20251001
- Successfully generated 164 Python solutions from GPT-4o using gpt-4o model
- Successfully generated 164 Python solutions from Gemini 2.5 Flash using gemini-2.5-flash
- All solutions saved to data/raw/ as CSV files with task_id, prompt, and ai_code columns

Dataset Preparation:

- Merged each AI model's 164 solutions with 164 human-written HumanEval-X canonical solutions
- Each merged CSV contains 328 rows (164 AI-labeled 0, 164 human-labeled 1), shuffled randomly
- 3 merged CSVs created: humaneval_claude_python_merged.csv, humaneval_gpt4o_python_merged.csv, humaneval_gemini15_python_merged.csv

AST Generation:

- Built tree-sitter grammars using compatible versions (Python v0.20.4, Java v0.20.1, C++ v0.20.3)
- Ran AST generation on all 3 merged CSVs — 0 rows dropped (all 328 rows parsed successfully for each file)
- Variable renaming applied before AST generation to normalize naming styles across human and AI code

Embedding Generation:

- Downloaded CodeBERT model (microsoft/codebert-base, ~500MB) from HuggingFace
- Generated 768-dimensional CLS token embeddings for both code and AST sequences
- Three embedding types created per sample: code_ (768-dim), ast_ (768-dim), combined_ (1536-dim)
- Created 5-fold stratified train/test splits for each of the 3 models (15 split directories total)

Planned Next Steps

- Run logistic regression classifier on all splits
- Analyze results and compare against paper's baseline numbers
- Push final code and results to GitHub
- Prepare professor demo and presentation